

THIRUMALASETTY HEMA

SENIOR SOFTWARE DEVELOPER

913-915-3535 | hemakumarithirumalasetty8@gmail.com

PROFESSIONAL SUMMARY

Senior Software Developer with 7+ years of experience building scalable full-stack web applications using Python, FastAPI, Django, React.js, JavaScript, TypeScript, SQL, and cloud technologies. Strong experience designing responsive frontend applications, reusable React components, secure REST APIs, microservices, database-driven business workflows, and enterprise integrations. Hands-on with PostgreSQL, MongoDB, AWS, Azure, Docker, CI/CD, automated testing, logging, monitoring, and production support. Also experienced with data engineering and selected AI/ML solutions where they add business value, including classification, NLP, and Generative AI. Comfortable working across requirements, design, development, testing, deployment, troubleshooting, and Agile delivery with cross-functional teams.

TECHNICAL SKILLS

Category	Technologies / Skills
Programming Languages	Python, SQL, JavaScript, TypeScript, Bash
Frontend	React.js, Redux Toolkit, React Hooks, HTML5, CSS3, JavaScript, TypeScript, Bootstrap, responsive UI, reusable components
Python / Backend	FastAPI, Django, Flask, REST APIs, Pydantic, SQLAlchemy, Celery, Microservices, authentication, API integrations
Databases	PostgreSQL, MySQL, SQL Server, MongoDB, Redis
Data Engineering	Pandas, NumPy, PySpark, Apache Spark, Kafka, Airflow, ETL/ELT, data validation
AI / Machine Learning	Scikit-learn, XGBoost, NLP, LLMs, RAG, LangChain, Hugging Face, embeddings, vector search, model evaluation
Cloud	AWS: EC2, S3, Lambda, API Gateway, RDS, IAM, CloudWatch; Azure: App Service, Functions, Blob Storage, Key Vault, Monitor
DevOps	Docker, Kubernetes, Git, GitHub, Jenkins, GitHub Actions, CI/CD, environment configuration, monitoring
Testing / Tools	PyTest, unittest, Postman, Swagger/OpenAPI, JIRA, Confluence, VS Code, PyCharm, Agile, Scrum, SDLC

PROFESSIONAL EXPERIENCE

Senior Software Developer - Python Full Stack / AI & ML

Wells Fargo, United States | Jan 2025 - Present

Project Description: Worked on enterprise banking applications supporting customer servicing, account and transaction workflows, operational dashboards, internal approvals, reporting, and data-driven business processes. The role focused primarily on full-stack application development, secure API integration, reliability, and maintainable enterprise software.

Frontend:

- Developed responsive React.js screens for customer profiles, account details, transaction search, operational work queues, approvals, exception handling, and reporting dashboards used by banking operations teams.
- Built reusable functional components with React Hooks, TypeScript, HTML5, CSS3, and Bootstrap, creating consistent layouts and behavior across customer servicing and transaction workflow modules.
- Implemented Redux Toolkit to manage shared application state such as authenticated user information, account data, workflow status, filters, and information reused across multiple screens.
- Developed complex forms and data grids with field validation, conditional controls, search, filtering, sorting, pagination, confirmation messages, and clear error feedback for business users.
- Integrated React applications with secured REST APIs, handling asynchronous requests, response mapping, loading indicators, empty states, API failures, and user-friendly error messages.
- Implemented role-based navigation and protected routes so users could view screens, actions, and workflow options according to their assigned permissions and business responsibilities.
- Improved frontend performance and maintainability by organizing reusable components, API service modules, utilities, and UI logic while reducing duplicate code and unnecessary rendering.

Backend:

- Designed and developed Python REST APIs and microservices using FastAPI and Django to support account servicing, transaction workflows, operational processing, reporting, and internal banking integrations.
- Created API endpoints with Pydantic request and response models, validation rules, standardized HTTP status codes, exception handling, structured logging, and Swagger/OpenAPI documentation.
- Developed service and repository layers to clearly separate API routing, business rules, validation, and database-access responsibilities, making backend services easier to maintain and test.
- Implemented authentication and authorization controls for secured services, validating user identity and permissions before processing sensitive account, transaction, and workflow operations.
- Designed and optimized PostgreSQL and SQL Server queries, joins, indexes, and data-access logic to improve performance for transaction processing, reporting, search, and operational workflows.
- Built asynchronous and scheduled Python processes for file processing, reconciliation, data validation, notifications, and high-volume operational workloads that did not require synchronous API processing.
- Integrated backend services with internal and third-party REST APIs using secure data-exchange patterns, request validation, response mapping, retry handling, logging, and production troubleshooting.

AI / Machine Learning:

- Developed machine-learning use cases for transaction and operational data, including classification and anomaly-detection workflows to help identify patterns and exceptions for further business review.
- Prepared and transformed structured datasets using Pandas and Scikit-learn, including data cleaning, feature engineering, train/test preparation, and model evaluation.
- Built NLP and Generative AI proof-of-concepts for enterprise knowledge search using embeddings, vector search, LangChain, and approved internal content.
- Evaluated model performance using precision, recall, F1 score, confusion matrices, and business-oriented threshold analysis before presenting results to stakeholders.

Cloud / DevOps:

- Deployed and supported Python APIs and application services on Azure App Service, configuring application settings and environment-specific values for development, testing, and production environments.
- Used Azure Functions for scheduled and event-driven processing such as background jobs, file-processing activities, notifications, and lightweight integration workloads outside the main application services.
- Used Azure Blob Storage for application files and data exchange, and Azure Key Vault to securely manage secrets, connection information, API credentials, and other sensitive configuration values.
- Containerized backend services with Docker and supported Git-based CI/CD pipelines that automated builds, unit testing, environment configuration, deployment steps, and controlled application releases.
- Used Azure Monitor, application logs, and deployment diagnostics to track service health, identify API or infrastructure failures, troubleshoot production issues, and support root-cause analysis.

Environment: Python, FastAPI, Django, React.js, Redux Toolkit, TypeScript, JavaScript, PostgreSQL, SQL Server, Pandas, Scikit-learn, LangChain, LLMs, RAG, Azure, Docker, Git, PyTest, Postman, REST APIs

Senior Software Developer - Python Full Stack

CDW, United States | May 2023 - July 2024

Project Description: Worked on an enterprise technology platform supporting product, customer, order, inventory, cloud-service, and operational information. The application used React-based user experiences and Python backend services to support enterprise search, reporting, workflow processing, and data-driven business operations.

Frontend:

- Developed React.js dashboards for product information, customer activity, inventory visibility, order status, enterprise search, and operational reporting used by business and support teams.
- Built domain-focused React components for catalog, customer, order, and inventory views, using custom Hooks and Redux Toolkit for cross-screen state coordination.
- Created data-heavy user experiences with configurable grids, faceted search, sorting, pagination, modal actions, and responsive layouts for large product and order datasets.
- Mapped Python service responses into frontend view models, normalizing product, inventory, and order data before presentation to business users.
- Applied workflow permissions at the UI level to control available actions for business, operations, and support personas.
- Reduced client-side rendering overhead through memoization, leaner state updates, and consolidation of common API utilities.

- Partnered with UX, QA, backend, and business teams to resolve usability gaps and data-mapping defects across catalog and fulfillment workflows.

Backend:

- Designed Python microservices using FastAPI to support product, customer, order, inventory, enterprise search, reporting, and operational workflow capabilities across the platform.
- Implemented FastAPI contracts with Pydantic and SQLAlchemy, documenting service behavior through Swagger/OpenAPI for downstream consumers.
- Structured backend modules around catalog, order, inventory, and customer domains so business rules and persistence concerns remained independently maintainable.
- Modeled transactional data in PostgreSQL and flexible catalog attributes in MongoDB, tuning retrieval paths for high-volume search and reporting.
- Processed product feeds and inventory updates asynchronously to keep long-running synchronization work outside user-facing request cycles.
- Enforced API access policies and audit-ready request tracing for internal integrations and enterprise service consumers.
- Created PyTest coverage for critical services and investigated production defects using API traces, database evidence, and runtime telemetry.

Cloud / DevOps:

- Ran product and order services on AWS EC2/ECS with configuration separated by deployment environment.
- Used S3 for inbound feeds and operational artifacts while Lambda handled lightweight event-driven processing.
- Published selected services through API Gateway and used IAM policies to restrict application-to-application access.
- Packaged services in Docker and maintained automated build, test, validation, and release stages in Git-based delivery pipelines.
- Tracked application health and performance through CloudWatch metrics, logs, and alerts to identify service degradation quickly.

Environment: Python, FastAPI, React.js, Redux Toolkit, TypeScript, JavaScript, PostgreSQL, MongoDB, AWS, Docker, Git, PyTest, Postman, Swagger/OpenAPI, REST APIs

Python Full Stack Developer

New York Community Bank, India | Sep 2021 - Mar 2023

Project Description: Worked on banking applications supporting customer profiles, account servicing, transaction workflows, payment operations, internal approvals, reconciliation, and operational reporting, with responsibilities across React frontend development and Python backend services.

Frontend:

- Developed React.js interfaces for customer profiles, account details, transaction history, payment workflows, service requests, and operational dashboards used by banking teams.
- Built reusable React components with Hooks, routing, Bootstrap, and responsive forms for account servicing, transaction search, and internal approval workflows.
- Integrated frontend screens with secured Python REST APIs and coordinated shared customer, account, and transaction data across multiple banking workflows.
- Implemented clear loading, validation, success, and error states for payment and servicing actions so operations users could track processing outcomes.
- Built data tables and dashboards with search, filtering, sorting, pagination, and status views for transaction review, exceptions, and reconciliation activities.
- Applied role-based UI controls so customer-service, operations, and supervisory users could access only the functions permitted for their responsibilities.
- Improved frontend maintainability by standardizing reusable components, API utilities, form validation, and common workflow patterns across banking modules.

Backend:

- Developed Django and FastAPI services for customer servicing, account information, transaction processing, payment workflows, reconciliation, notifications, and operational reporting.
- Defined secure API contracts with authenticated requests, validated payloads, consistent response structures, exception handling, and audit-friendly transaction processing.
- Modeled customer, account, transaction, payment, and workflow data using PostgreSQL and MongoDB based on structured transactional and flexible operational requirements.

- Built Python ETL processes for transaction and reconciliation files, including validation, cleansing, transformation, exception identification, and database loading.
- Implemented scheduled jobs for reconciliation, operational reporting, notifications, and processing of banking workloads outside synchronous user requests.
- Integrated internal banking services and external interfaces through controlled API and file-based integration routines with retry handling and traceable failures.
- Optimized data-access routines, tested APIs with PyTest and Postman, and investigated production issues using logs, database queries, and transaction traces.

Cloud / DevOps:

- Hosted Python banking services on AWS EC2 with relational workloads managed in Amazon RDS and environment-specific application configuration.
- Used Amazon S3 for inbound transaction files, reconciliation outputs, reports, and controlled data exchange between application processes.
- Used AWS Lambda for lightweight scheduled and event-driven processing such as file validation, notifications, and operational background tasks.
- Applied IAM permissions to application resources and used CloudWatch logs and metrics to monitor services, investigate failures, and support production troubleshooting.
- Delivered containerized services through Docker and Jenkins/Git pipelines with automated testing and controlled deployment stages across environments.

Environment: Python, Django, FastAPI, React.js, JavaScript, PostgreSQL, MongoDB, AWS, Docker, Jenkins, Git, PyTest, Postman, REST APIs

Software Developer - Python

Darwinbox, India | Aug 2019 - Sep 2021

Project Description: Worked on an HR technology platform supporting employee information, onboarding, attendance, leave, organizational workflows, HR operations, and reporting. Responsibilities included Python backend development, React-based user interfaces, workflow automation, and cloud deployment support.

Frontend:

- Developed responsive React.js screens using JavaScript, HTML5, CSS3, and Bootstrap for employee profiles, onboarding activities, attendance, leave requests, approvals, and HR reporting.
- Created reusable UI controls for employee data entry, search, approval workflows, dashboards, and HR operations with a focus on simple day-to-day usability.
- Connected React screens to Python APIs and presented clear validation, success, and failure feedback for employee updates, leave requests, and workflow actions.
- Organized multi-page navigation for employee self-service, manager approvals, HR administration, and reporting as the platform expanded across HR modules.
- Resolved responsive-layout and browser-compatibility issues to keep employee and HR workflows consistent across supported desktop environments.
- Refactored common UI helpers, form logic, and API interaction code to improve reuse and maintainability as additional HR features were introduced.
- Worked with QA and backend developers during defect triage to isolate UI, API, workflow, and employee-data issues across HR modules.

Backend:

- Built Django and Flask applications supporting employee master data, onboarding, attendance, leave management, approval workflows, notifications, and HR reporting.
- Developed Python utilities for employee-data imports, validation, cleansing, transformation, and workflow automation used by HR and operations teams.
- Worked with SQL Server, MySQL, PostgreSQL, and MongoDB for employee, attendance, leave, workflow, and reporting data, writing queries for operational screens and recurring reports.
- Introduced structured logging and defensive exception handling to make API, workflow, and employee-data processing failures easier to diagnose.
- Automated recurring batch workloads for employee imports, attendance processing, notifications, and report generation, reducing manual HR operational steps.
- Implemented integrations with external and internal HR-related services using controlled request handling, validation, and failure recovery for dependable data exchange.

- Contributed unit tests, code reviews, query tuning, release validation, and production fixes as part of the team's regular engineering delivery cycle.

Cloud / DevOps:

- Supported AWS EC2 and RDS deployments for Python-based HR application services and database workloads across development and test environments.
- Used Amazon S3 for employee import files, generated reports, processing outputs, and documents used by HR workflows.
- Maintained least-privilege IAM permissions and environment-specific configuration for cloud-hosted application services and supporting resources.
- Reviewed CloudWatch logs during incidents to trace API, workflow, background-processing, and deployment failures to their source.
- Used Docker, Git, and Jenkins to package application services and support repeatable build, test, and deployment practices.

Environment: Python, Django, Flask, React.js, JavaScript, HTML5, CSS3, SQL Server, MySQL, PostgreSQL, MongoDB, AWS, Docker, Jenkins, Git, REST APIs

EDUCATION

Master's in Computer Science and Technology Management

Avila University, Kansas City, Missouri, USA

Bachelor's in Computer Science

Andhra Pradesh, India